CSCE 410 P2 Report: Multilevel Priority Scheduling

Zayd Nadir

1. Submission information

a. Repository URL: https://github.com/zaydn4321/333006458-xv6-csce410-p1-p2

The P1 code is tagged `P1-submission`. Everything after that tag is P2.

b. Late-day tokens: No, none used.

2. Resources used

a. Which resources did you leverage while developing your solution?

I used the xv6 book (the scheduling chapter and the sleep/wakeup section), the OSTEP scheduling chapters, the xv6 source code, and Claude Code. For the bonus plan I only used the two books and the source code like the handout says.

b. How were they useful?

The xv6 book and the source code showed me every place a process becomes RUNNABLE and how the scheduler and sched() hand p->lock back and forth, which is what my queue design is based on. I used Claude Code to talk through the lock order, to help write and debug the starvation prevention, the dispatch trace and schedtest, and to run the tests over and over in QEMU. This report was also written with the help of Claude Code. I reviewed and tested everything it produced.

c. What is your estimate for how much time these resources saved you?

About 6 to 8 hours. Most of that came from debugging faster and from not having to run and check the long test sessions by hand.

3. Queue design and implementation

Where NB_PRIORITY_LEVELS is defined. I put it in `kernel/param.h` next to the other limits like NPROC. Both the kernel and the user test programs include param.h, so prtest and schedtest always use the same number of levels as the kernel instead of hardcoding it. It is inside `#ifndef`, so `make clean; make NB_PRIORITY_LEVELS=n` changes it without editing the file. I added that variable to the Makefile the same way as the `DBG=0` option from P1. There is an `#error` if it is less than 2, since the default priority 1 has to exist. DEFAULT_PRIORITY (1) and STARVE_TICKS (10) are in the same file.

Queues. There is one FIFO queue per priority level. Each queue is a linked list through a new `qnext` field in struct proc, with a head and tail array in proc.c. Nothing gets allocated because a process is only ever in one queue, so the link can live in its own proc slot. Push adds to the tail and pop takes from the

head of the first non-empty level, starting at 0. Since a process that gets preempted goes to the back of its level, processes with the same priority take turns (round robin).

Synchronization. I used one spinlock, `rq.lock`, for all the queues. It is always taken last. Callers can be holding p->lock (and tickslock, through clockintr() calling wakeup()), and nothing else is acquired while holding it. The scheduler pops a process, releases rq.lock, and then acquires p->lock. That is safe because a popped process is RUNNABLE and in no queue, and nothing else takes a process out of RUNNABLE (kill only sets killed, wakeup only looks at SLEEPING processes). The scheduler panics if that ever isn't true. I used one lock instead of one per level because there is only one CPU, the critical sections are just a few pointer changes, and it lets rq_check() see all the levels at once.

make_runnable(). Every RUNNABLE transition goes through this function, which sets the state and pushes the process onto its queue. userinit, kfork, yield, wakeup and kkill all use it, and those are the only places in the kernel that set RUNNABLE.

Starvation prevention. rq_push() saves the current tick in a new `qtime` field. Since the queues are FIFO, the head of each level is the one that has waited the longest there. When the scheduler pops, if any head has waited STARVE_TICKS or more, the oldest one runs instead of the highest priority one. Its priority doesn't change, so it goes back into the same queue after. This keeps queue membership matching priority and keeps it a plain priority scheduler instead of a feedback queue. A low priority process can still wait, but not forever. One problem is that it can't take tickslock to read ticks, because clockintr() holds tickslock while it takes p->lock and rq.lock, so that would be the opposite lock order. I read ticks with an atomic load instead.

Unit checks. rq_check() goes through every queue and proc[] and reports cycles, a process queued twice, a queued process that isn't RUNNABLE, a process in the wrong queue for its priority, a wrong tail pointer, and a RUNNABLE process that isn't in any queue. These are the correctness rules from the handout. User programs call it with the P1 debugctl syscall (`DBGCTL_SCHEDCHK`), and it also runs on Ctrl-P (procdump), which now prints each process's priority too.

System calls. setpriority (24) and getpriority (25) are added in syscall.h, syscall.c, sysproc.c, usys.pl and user.h. setpriority returns -1 for anything outside 0 to NB_PRIORITY_LEVELS-1 and doesn't change the priority. A running process isn't in any queue, so changing its priority just changes the field. If the value changed it calls yield(), which puts it in the new queue. Setting the same value doesn't yield.

4. Process lifecycle

Slot reuse. allocproc() and freeproc() both reset priority to 1, qnext to 0 and qtime to 0, so a reused slot never has old scheduling data.

fork. allocproc() sets the default, then kfork() copies the parent's priority before the child becomes RUNNABLE, so the child starts in the parent's queue.

First process. userinit() uses make_runnable() like everything else, so init starts in the priority 1 queue.

Sleep, wakeup and kill. A sleeping process isn't in any queue. wakeup() and kkill() both call make_runnable() while holding p->lock, so a process woken up by kill goes into the queue for its own

priority. schedtest checks this case.

exit. A running process is never in a queue and a ZOMBIE never becomes RUNNABLE, so exit didn't need any queue changes.

Problems.

The comment I first wrote in param.h said to use `make CFLAGS+=-D...` to change the number of levels. While writing this report I found out that setting CFLAGS on the command line replaces all of the Makefile's flags, so the kernel wouldn't build right. I added the `NB_PRIORITY_LEVELS=` make variable instead.

During the bonus session, usertests got stuck once in sbrkfail and timed out. I couldn't make it happen again. It passed 6 more times on that same commit and 8 times on the final kernel, and sbrkfail alone passed 15 times. I went through every state change and the lock order and didn't find a lost wakeup. One thing that would look like this is a kill not waking up a sleeping child. sbrkfail has 10 children that each sleep about 100 seconds in pause(1000), so that would take longer than the 600 second timeout. That's why schedtest tests the kill wakeup path by itself.

5. Initial test coverage

All tests run with CPUS=1, which is now the Makefile default.

usertests. I ran `CPUS=1 python3 test-xv6.py -q usertests` before changing anything and again on the new scheduler to make sure nothing else broke. It prints ALL TESTS PASSED.

prtest. `user/prtest.c` prints OK or FAIL for each check and exits with 1 if anything failed. It checks that:

- the default priority is 1
- -1, NB_PRIORITY_LEVELS and 1000 return -1 and don't change the priority
- every valid level can be set and read back, and setting the same value works
- a forked child gets its parent's priority
- with two children forked lowest priority first, the priority 0 child finishes first, even though the order they were forked in would let the low one go first
- rq_check is clean while the children are queued and at the end

How I checked it worked. Each check prints its own result, and rq_check prints exactly what it found (for example `pid 5 prio 2 in queue 1`), so a failure shows what broke.

6. Increased test coverage

Observing the scheduler. The P1 dprintf in the scheduler prints on every context switch, which is too slow for the hot path. Instead, rq_pop() writes a small entry into a 512 entry ring buffer in memory with the tick, pid, priority, and whether it was picked because it was starving. It does this while it already holds rq.lock, so it is just a few stores and no extra lock. `debugctl(DBGCTL_SCHEDLOG, buf)` copies it out and clears it. I added `dbg trace` to print it, and `dbg trace > file` saves it to a file. The tests also read this log to check the order things ran in.

Starvation test. I added test_starve to prtest. The parent spins at priority 0 for 5 times STARVE_TICKS without sleeping. The lowest priority child has to run before the parent finishes, and the trace has to show it was picked as starving.

schedtest. `user/schedtest.c` has the harder tests:

- `test_rr` - 3 children that use the CPU at the lowest level. From the trace, each one has to run several times and all three have to start before any of them finishes. That shows they take turns instead of running one at a time.
- `test_levels` - one child per level, forked lowest first. They have to finish in level order (0123 with 4 levels).
- `test_killwake` - a lowest priority child sleeping in pause(1000) gets killed while a priority 1 process spins. rq_check has to be clean right after the kill, and the killed child has to exit before the spinning one finishes.
- `test_churn` - 8 children each do 20 rounds of changing priority, sleeping and forking. The parent runs rq_check every tick for 30 ticks and it has to find nothing. Each child checks that its priority is the same after sleeping and that its own child gets the same priority.

Results. schedtest passed 10 times in a row (about 11 seconds each). prtest and schedtest both pass with NB_PRIORITY_LEVELS set to 2, 4 and 8. usertests passes on the final kernel. To make sure the starvation tests actually test something, I turned off the starvation check in rq_pop() for a run. prtest's test_starve and schedtest's test_killwake both failed, and passed again when I put it back.

Problems.

The first version of schedtest had each child spin a fixed number of loops. That took less than one tick on my machine, so each child only ran once or twice and the round robin check failed. A fixed loop count also depends on how fast the machine is.

I changed it to count ticks of the child's own time, only counting when uptime() went up by exactly 1. That made the test hang. The timer preempts the running process every tick, so with 3 children taking turns each one only ever saw uptime jump by 3.

Now work(n) counts every change in uptime(). Each change means the process just finished a turn, no matter what ran in between. The test doesn't depend on machine speed anymore.

The tests also used priority 2 at first, which doesn't exist with 2 levels. They now use NB_PRIORITY_LEVELS - 1 for the lowest and 0 for the highest, so they work with any number of levels.

Commit note. The commit that added the dispatch trace (`ba7a51b`) was pushed with the same message as the starvation prevention commit before it (`99afcb3`). The second one is the trace.

7. Source code added or modified

New files:

- `user/prtest.c` - the initial tests
- `user/schedtest.c` - the harder tests

- `bonus-report-part1.txt`, `bonus-report-part2.txt` - the bonus reports

Modified files:

- `Makefile` - CPUS defaults to 1, added _prtest, _schedtest and the `NB_PRIORITY_LEVELS=` option
- `kernel/param.h` - NB_PRIORITY_LEVELS, DEFAULT_PRIORITY and STARVE_TICKS
- `kernel/proc.h` - priority, qnext and qtime in struct proc
- `kernel/proc.c` - the queues, make_runnable(), the new scheduler, starvation prevention, the dispatch log, rq_check(), setpriority() and getpriority()
- `kernel/defs.h` - declarations for the new functions
- `kernel/syscall.h`, `kernel/syscall.c`, `kernel/sysproc.c` - the setpriority and getpriority syscalls
- `user/usys.pl`, `user/user.h` - user side of the new syscalls
- `kernel/debug.h`, `kernel/debug.c` - `DBGCTL_SCHEDCHK` and `DBGCTL_SCHEDLOG`
- `user/dbg.c` - `dbg trace`

I also removed `.github/workflows`.